The Transformer, end to end

Vaswani et al., "Attention Is All You Need" (2017) - base model, d_model = 512, 8 heads, 6 layers

Worked example: "The cat sat down" -> "Le chat est assis"

NOTATION KEY

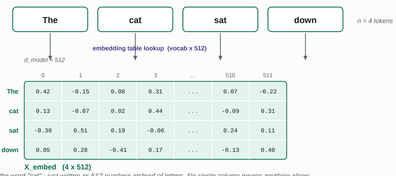
- **Encoder-side tensor** — one row per source token
- **Decoder-side tensor** — one row per target token
- **Learned weight matrix** — trained by backpropagation
- **Fixed, not learned** — positional encoding, causal mask
- **Encoder memory** — computed once, read 6 times

(+) = elementwise add (residual) x = matrix multiply shapes written as (rows x columns) grey cells = masked to -INF

ENCODER TRACK "The cat sat down"

1 Tokenize and embed the source

Each word becomes one row of 512 learned numbers



	0	1	2	3	...	510	511
The	0.42	-0.15	0.08	0.31	...	0.07	-0.22
cat	0.13	-0.87	0.02	0.44	...	-0.09	0.31
sat	-0.30	0.51	0.19	-0.06	...	0.24	0.11
down	0.05	0.28	-0.41	0.17	...	-0.13	0.40

X_embed (4 x 512)

Row 2 IS the word "cat" - just written as 512 numbers instead of letters. No single column means anything alone; the 512 numbers only carry meaning together, as one point in 512-dimensional space. Same table for every occurrence of a word. 512 is a design choice made by the authors, not derived from anything - every other width in the network follows from it.

2 Add positional encoding

Sine and cosine waves give each row a sense of where it sits

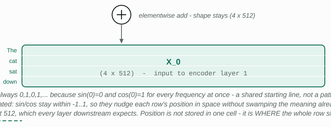
d_model = 512

	0	1	2	3	...	510	511
pos 0	0.000	1.000	0.000	1.000	...	0.0000	1.0000
pos 1	0.841	0.540	0.823	0.569	...	0.0001	1.0000
pos 2	0.909	-0.416	0.938	-0.348	...	0.0002	1.0000
pos 3	0.141	-0.990	0.239	-0.971	...	0.0003	1.0000

PosEnc (4 x 512) - fixed, computed from a formula

```
PE(pos, 2i)   = sin( pos / 10000^(2i/512) )
PE(pos, 2i+1) = cos( pos / 10000^(2i/512) )
i = 0 -> fastest wave (cols 0,1)
i = 255 -> slowest wave (cols 510,511)
```

fast / medium / slow - combined, unique per position



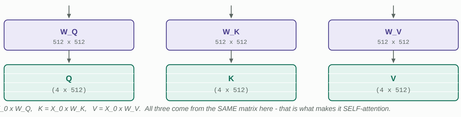
Row 0 of PosEnc is always 0,1,0,1,... because sin(0)=0 and cos(0)=1 for every frequency at once - a shared starting line, not a pattern that continues. Added, not concatenated: sin/cos stay within -1..1, so they nudge each row's position in space without swamping the meaning already there, and the width stays at 512, which every layer downstream expects. Position is not stored in one cell - it is WHERE the whole row sits in space.

ENCODER LAYER 1

3 Encoder self-attention (layer 1)

Every token looks at every other token and pulls in what it needs

3a Three projections



Q = X_0 x W_Q, K = X_0 x W_K, V = X_0 x W_V. All three come from the SAME matrix here - that is what makes it SELF-attention.

3b Split into 8 heads - slice the 512 columns into 8 chunks of 64 (rows are untouched)

each head gets Q_h, K_h, V_h of shape (4 x 64)

3c Scaled dot-product attention - shown for head 1

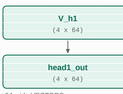
raw scores Q x K^T (4x64)x(64x4) -> (4x4), then / sqrt(64) = / 8

	The	cat	sat	down
The	4.1	2.0	1.2	0.9
cat	2.0	6.3	3.1	1.4
sat	1.2	3.1	5.8	2.2
down	0.9	1.4	2.2	4.7

after softmax (per row) each row now sums to 1

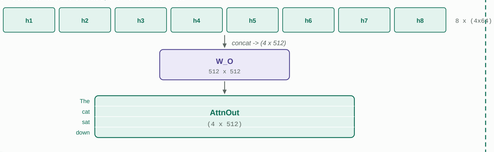
	The	cat	sat	down
The	0.55	0.20	0.15	0.10
cat	0.10	0.55	0.25	0.10
sat	0.08	0.24	0.53	0.15
down	0.11	0.15	0.24	0.50

weighted sum with V (4x4) x (4x64) -> (4x64)



Row "cat" of head1_out = 0.10 x V_r[The] + 0.55 x V[cat] + 0.25 x V[sat] + 0.10 x V_r[down]. Those four weights scale four 64-wide VECTORS, and every one of the 64 columns gets its own weighted sum using those same four weights - so the output row is still 64 numbers wide, not 4. Divide by sqrt(64) = 8 because summing 64 products makes raw scores grow large, which pushes softmax too peaked and shrinks gradients.

3d / 3e Concatenate the 8 heads, then one more linear



4 Add & norm, feed-forward, add & norm

Closing out encoder layer 1

R1 = X_0 + AttnOut (4 x 512)

LayerNorm *normalises ACROSS the 512 columns of ONE row, separately for every row (this is not batch norm)*

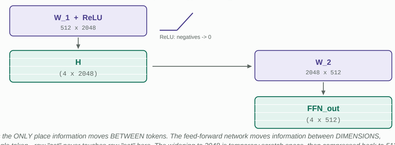
R1[cat]	2.30	0.87	-1.10	4.02	-0.55	1.80	...
(x - mean) / std	0.62	-0.20	-1.34	1.62	-1.02	0.33	...
x gamma + beta	0.78	-0.28	-1.34	2.03	-0.87	0.33	...

mean = 1.223 (of all 512 in this row)
std = 1.731 (of all 512 in this row)
gamma, beta: two learned 512-vectors, shared by every row in this layer

one row in -> mean 0, spread 1 -> rescaled by learned gamma and beta -> one row out, still 512 wide

Z1 (4 x 512)

Feed-forward network - applied to each row independently, no mixing between tokens



Attention is the ONLY place information moves BETWEEN tokens. The feed-forward network moves information between DIMENSIONS, within a single token - row "cat" never touches row "sat" here. The widening to 2048 is temporary scratch space, then compressed back to 512.

R2 = Z1 + FFN_out (4 x 512)

LayerNorm

The cat sat down

X_1 (4 x 512) - output of encoder layer 1

ENCODER LAYER 2 — *own separate W_Q, W_K, W_V, W_O, W_1, W_2, g, b* — X_2

layers 3, 4, 5 - identical structure, all with their own weights

ENCODER LAYER 6 — *same structure again* — X_6

Each layer takes the PREVIOUS layer's output, never X_0 again: X_0 -> L1 -> X_1 -> L2 -> X_2 -> ... -> L6 -> X_6.

5 Encoder output - the memory

Computed once. Every decoder layer reads it.

MEMORY = X_6

(4 x 512) - one row per SOURCE token

Memory[The] (1 x 512) | Memory[cat] (1 x 512) | Memory[sat] (1 x 512) | Memory[down] (1 x 512)

After 6 layers, Memory[cat] no longer means "cat" generically - it means "cat, as used in this particular sentence", having absorbed context from every other word.

DECODER TRACK "Le chat est assis"

6 Tokenize and embed the target, shifted right

The decoder is fed one step behind what it must predict

target (what we want):	Le	chat	est	assis	"assis" is never fed in
decoder input (shifted):	<BOS>	Le	chat	est	

Without the shift, position 2 would already contain the word it is being asked to predict - the model would learn to copy, not to translate.

	0	1	2	3	...	510	511
<BOS>	0.09	0.34	-0.27	0.12	...	0.19	-0.05
Le	-0.22	0.61	0.14	-0.38	...	0.02	0.27
chat	0.47	-0.11	0.36	0.20	...	-0.16	0.08
est	0.18	0.25	-0.09	0.53	...	0.31	-0.20

Y_embed (4 x 512)

same PosEnc formula, positions counted within the DECODER's own sequence

d_model = 512

	0	1	2	3	...	510	511
pos 0	0.000	1.000	0.000	1.000	...	0.0000	1.0000
pos 1	0.841	0.540	0.823	0.569	...	0.0001	1.0000
pos 2	0.909	-0.416	0.938	-0.348	...	0.0002	1.0000
pos 3	0.141	-0.990	0.239	-0.971	...	0.0003	1.0000

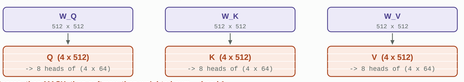
PosEnc (4 x 512) - same formula, same values

DECODER LAYER 1

7 Masked self-attention (decoder layer 1)

Same machinery as step 3, plus one causal mask

7a / 7b Q, K, V from Y_0, then split into 8 heads of 64 (identical to steps 3a-3b)



7c Scores, then MASK, then softmax, then weighted sum - head 1

raw scores / 8 — Q x K^T then / sqrt(64)

	BOS	Le	chat	est
BOS	5.2	1.1	0.7	1.9
Le	1.1	6.0	2.4	1.2
chat	0.7	2.4	5.5	3.0
est	1.9	1.2	3.0	5.9

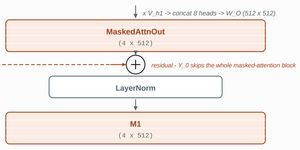
apply causal mask — strictly above diagonal -> -INF

	BOS	Le	chat	est
BOS	5.2	-INF	-INF	-INF
Le	1.1	6.0	-INF	-INF
chat	0.7	2.4	5.5	-INF
est	1.9	1.2	3.0	5.9

softmax per row — e^(-INF) = 0 exactly

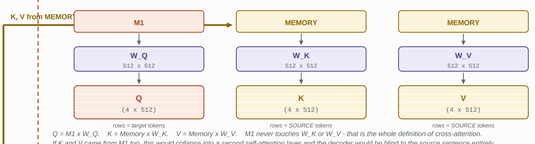
	BOS	Le	chat	est
BOS	1.00	0.00	0.00	0.00
Le	0.12	0.88	0.00	0.00
chat	0.15	0.60	0.25	0.00
est	0.14	0.10	0.26	0.50

The diagonal is NOT masked - a token may always attend to itself. Masked cells are strictly above the diagonal (column index > row index). Why -INF and not zero: softmax must drive these to EXACTLY 0 probability. Any finite value would still leak a whisper of a future token into the sum. Why mask at all: the decoder computes all positions in parallel, so without a mask "chat" could simply read "est" and copy it. Masking makes parallel training behave like sequential generation - matching the real condition at inference, where later tokens genuinely do not exist yet.



8 Cross-attention - where the two tracks meet

Queries from the decoder, Keys and Values from the encoder

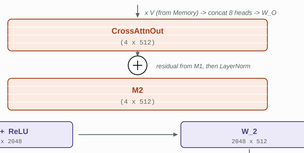


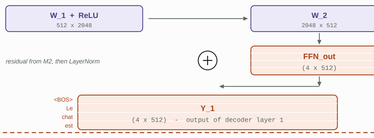
Q = M1 x W_Q, K = Memory x W_K, V = Memory x W_V. M1 never touches W_K or W_V - that is the whole definition of cross-attention. If K and V came from M1 too, this would collapse into a second self-attention layer and the decoder would be blind to the source sentence entirely. Memory is computed once, but each of the 6 decoder layers projects it with its OWN W_K and W_V.

cross-attention weights (target rows x source columns)

no mask here - the entire source sentence was already known when Memory was built

	The	cat	sat	down
BOS	0.05	0.70	0.15	0.10
Le	0.08	0.62	0.20	0.10
chat	0.10	0.12	0.63	0.15
est	0.06	0.09	0.20	0.65





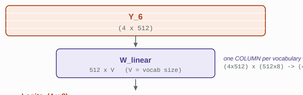
DECODER LAYER 2 — *masked self-attn -> add&norm -> cross-attn -> add&norm -> FFN -> add&norm* — Y_2

layers 3, 4, 5 - each with its own weights, each re-reading the same Memory

DECODER LAYER 6 — *same structure again* — Y_6

9 Final linear and softmax

Turn 512 numbers per position into a probability for every word



Logits (4 x 8)

raw, unbounded scores

	<BOS>	<EOS>	Le	chat	est	assis	sur	tapis
<BOS>	-1.2	-3.0	4.8	0.4	-0.9	-1.7	-2.1	-2.4
Le	-1.8	-2.6	0.2	5.1	1.0	-0.7	-1.9	-1.1
chat	-2.4	-1.9	-0.6	0.7	5.6	1.2	0.9	0.3
est	-2.0	-0.4	-1.1	0.2	0.8	5.4	1.1	0.6

Output probabilities (4 x 8)

softmax per row - each row sums to 1

	<BOS>	<EOS>	Le	chat	est	assis	sur	tapis
<BOS>	0.00	0.00	0.94	0.02	0.01	0.01	0.01	0.01
Le	0.00	0.00	0.01	0.93	0.03	0.01	0.01	0.01
chat	0.00	0.01	0.02	0.03	0.85	0.04	0.03	0.02
est	0.00	0.02	0.01	0.02	0.03	0.86	0.04	0.02

Read the highlighted row: given <BOS>, Le, chat so far, the model is 85% confident the next word is "est". Logits[chat, est] = Y_6[chat] . W_linear[:, est] - a dot product of 512 pairs, measuring how well that row lines up with the learned "est" direction.

9d At inference, the decoder runs once per output word

pass 1	<BOS>	-> Le
pass 2	<BOS>, Le	-> chat
pass 3	<BOS>, Le, chat	-> est
pass 4	<BOS>, Le, chat, est	-> assis
pass 5	<BOS>, Le, chat, est, assis	-> <EOS> -> stop

Each pass is a COMPLETE trip through steps 6-9, with one more input tile than the pass before.

Only the LAST row's argmax is used; earlier rows are recomputed, discarded.

argmax of the last position's probability row -> appended to end of decoder input -> full sequence re-run

INFERENCE ONLY - loop back to

During TRAINING none of this looping happens. The full correct shifted sequence is fed in once and scored at every position simultaneously - teacher forcing. That is precisely WHY the mask in step 7 has to exist: it recreates "only see what came before" inside a single parallel pass.

d_model = 512 | 8 heads x 64 dims | FFN inner width 2048 | 6 encoder layers, 6 decoder layers | every attention block uses softmax(Q K^T / sqrt(64)) V

Attention moves information BETWEEN tokens. Feed-forward moves it BETWEEN dimensions. Residuals keep the original signal alive through all 6 layers.